

EX1: 8 QUEEN PROBLEM

```
1 N = 8
2 def print_solution(board):
3     for row in board:
4         print(" ".join("Q" if col else "." for col in row))
5     print()
6 def is_safe(board, row, col):
7     for i in range(row):
8         if (
9             board[i][col]
10            or (col - (row - i) >= 0 and board[i][col - (row - i)])
11        ):
12            return False
13    return True
14 def solve_n_queens(board, row):
15     if row == N:
16         print_solution(board)
17         return True
18     found = False
19     for col in range(N):
20         if is_safe(board, row, col):
21             board[row][col] = 1
22             found = solve_n_queens(board, row + 1) or found
23             board[row][col] = 0
24     return found
25 board = [[0] * N for _ in range(N)]
26 solve_n_queens(board, 0)
27
```

OUTPUT:

Output									
Q	.	.	.	.	.	.	.	.	.
.	.	Q	.	.	.	.	.	.	.
.	.	.	.	.	Q	.	.	.	.
.	.	.	.	.	.	.	.	Q	.
.	.	.	.	.	.	.	Q	.	.
.	.	.	Q	.	.	.	.	.	.
.	Q	.	.	.	.	.	.	.	.
.	.	.	.	Q	.	.	.	.	.
Q	.	.	.	.	.	.	.	.	.
.	.	Q	.	.	.	.	.	.	.
.	.	.	.	.	Q	.	.	.	.
.	.	.	.	.	.	.	.	Q	.
.	.	.	.	.	.	.	.	Q	.
.	.	.	.	Q	.	.	.	.	.
.	Q	.	.	.	.	.	.	.	.
.	.	.	Q	.	.	.	.	.	.
Q	.	.	.	.	.	.	.	.	.
.	.	Q	.	.	.	.	.	.	.
.	.	.	.	.	Q	.	.	.	.
.	.	.	.	.	.	.	.	Q	.
.	.	.	.	.	.	.	.	Q	.
.	.	.	.	Q	.	.	.	.	.
.	.	.	Q	.	.	.	.	.	.
.	Q	.	.	.	.	.	.	.	.